1 THE PROBLEM

Group projects are hard.
Finding one free hour is harder.

Four teammates, four different timetables. Our program collects everyone's free hours, finds the time when all of them are free, and sends the meeting to each person where they actually read messages.

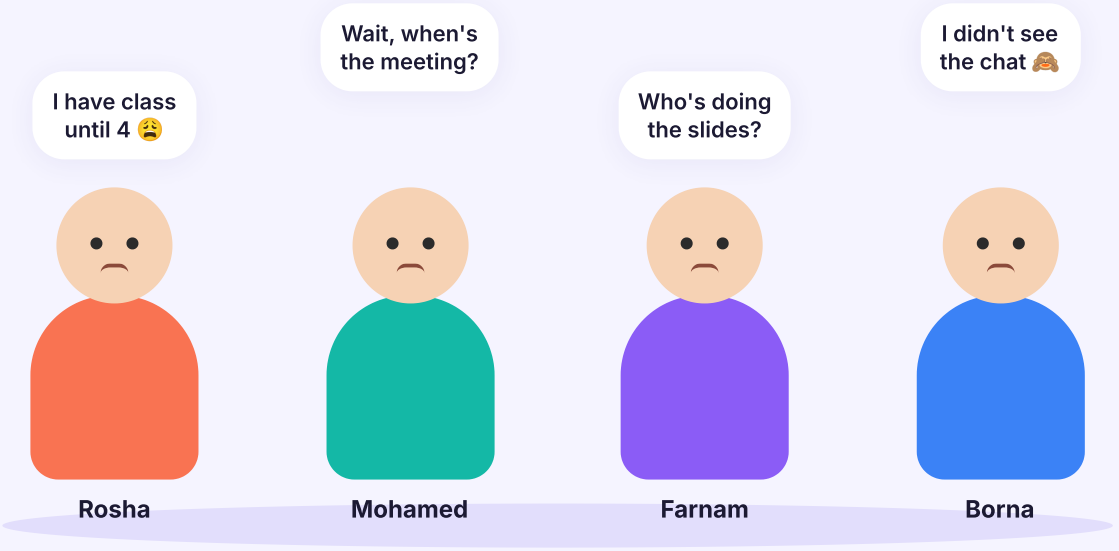
⏰ **"When is everyone free?"** → we compare everyone's hours automatically

💬 **"I never saw the message."** → email, Telegram, WhatsApp or SMS, each person picks

🔁 **"Did we already send this?"** → the same meeting is never sent twice

How to read this walkthrough

💻 **Left:** what our program does today, with the real messages from the terminal.
📱 **Right:** how the same step would look if it were a real app.
The schedules are examples, but every result is calculated by our `scheduler.py`.
🚧 / 🔮 Pages marked **In progress** or **Coming next** show what we're adding next.



Rosha, Mohamed, Farnam and Borna: one team, one project, four timetables

2 CREATING A GROUP

Start a group, then add everyone, step by step

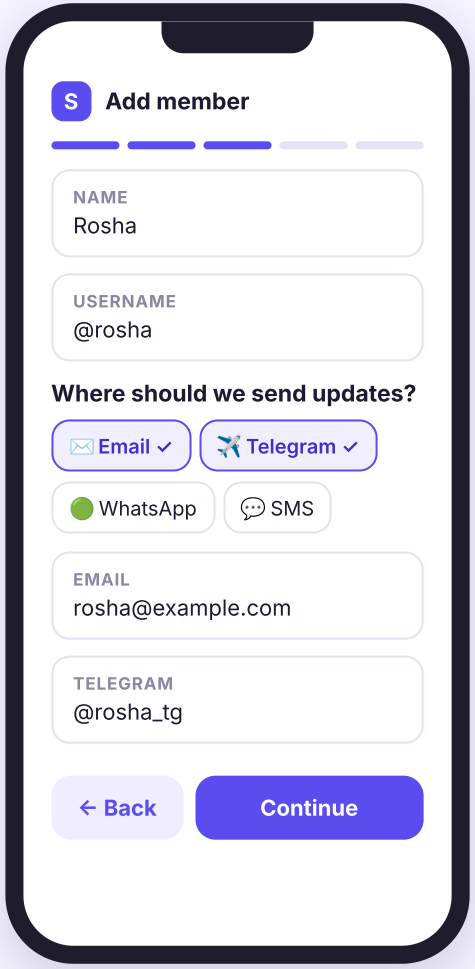
Rosha creates the group and adds each member: name, username, notification platforms and contact info. At every question `b` goes one step back and `0` cancels.

```
python main.py

===== Smart Group Project =====
1. Create Group   2. Add Member   3. Manage Availability
4. Suggest Meeting Time   5. Save   6. Load   0. Exit
Choose an option: 1
Group name (0=Cancel): University Project
Group 'University Project' created successfully!

Choose an option: 2
Enter member name (0=Cancel): Rosha
Enter username (b=Back, 0=Cancel): rosha
Notification platforms: 1.Email  2.Telegram  3.WhatsApp  4.SMS
Choose (e.g. 1,2; b=Back, 0=Cancel): 1,2
Email (b=Back, 0=Cancel): rosha@example.com
Telegram username (b=Back, 0=Cancel; Enter=Skip): rosha_tg
   ... Telegram verification (next page) ...
Review: Rosha (@rosha)
Active notifications: email, telegram
Save member? (y/n, b=Back, 0=Cancel): y
Member Rosha (@rosha) added successfully! ID: 1
```

💻 Our program today · `sgp/cli/menu.py`



📱 As a real app

CHECKED BY OUR CODE

✅ Name: 1–20 English letters
✅ Username can't be taken twice
✅ Email format, like `name@example.com`

5 STEPS

Name → Username → Platforms → Contact info → Review

SAVED AS

`Member(id=1, name="Rosha", username="rosha")`

3 CONNECTING TELEGRAM

Prove it's really your Telegram

Anyone could type someone else's username, so we don't trust it. The member sends a one-time code to our bot, and the program only accepts it if the right person sends it from a private chat.

```
python main.py

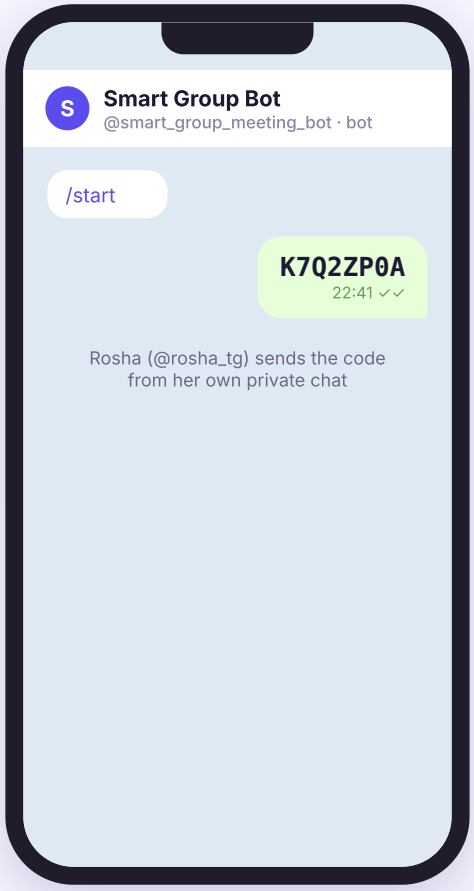
Open https://t.me/smart_group_meeting_bot and press Start.
Connection code: K7Q2ZP0A
Send the code to the bot. Verification may wait up to 5 minutes.

Waiting for Telegram verification...
You have 5 minutes to send the code.
Press 0 to Cancel or b to Back while waiting.
   ... checking new messages every 3 seconds ...
Verification successful!
Telegram connected successfully!
```

💻 **Our program today** · `telegram_verification.py`

HOW THE CODE IS MADE

8 random characters from A–Z and 0–9, created with Python's `secrets` module, so it can't be guessed. Old messages are skipped, so an old code never works.



📱 **Rosha's Telegram**

THE BOT ACCEPTS THE CODE ONLY IF...

✅ **It's the exact code**
K7Q2ZP0A, extra spaces don't matter

✅ **The username matches**
@rosha_tg, upper/lower case ignored

✅ **It's a private chat**
not a group chat

⛔ **Someone else sends it**
→ ignored

⛔ **5 minutes pass**
→ "Verification code expired!"

4 ADDING FREE HOURS

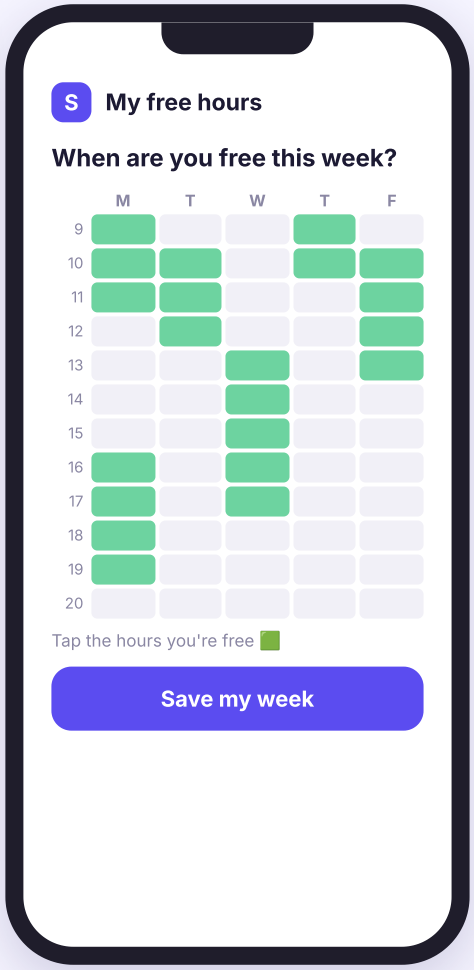
Everyone adds when they're free

Each member enters their free hours for each day, in whole hours like 9-12. Wrong input (like 25-30 or 12-9) is rejected, and overlapping slots are joined automatically.

```
python main.py

Choose an option: 3
Member name (b=Back, 0=Cancel): Rosha
===== Availability: Rosha =====
1. View   2. Add / Replace Day   3. Delete Day
Choose: 2
Day (b=Back, 0=Cancel): wednesday
Time slot (e.g. 9-12; Enter=Finish, ...): 13-15
Time slot added.
Time slot (e.g. 9-12; Enter=Finish, ...): 14-18
Time slot added.
Time slot (e.g. 9-12; Enter=Finish, ...):
Proposed Wednesday: [(13, 18)]
Save this day? (y/n, b=Back, 0=Cancel): y
Availability saved!
Waiting for other members to complete their availability.
```

💻 **Our program today** · Manage Availability



📱 **Rosha's week as an app**

OVERLAPPING SLOTS ARE MERGED

13–15

14–18

= 13–18

`merge_intervals()` sorts the slots and joins the ones that overlap or touch.

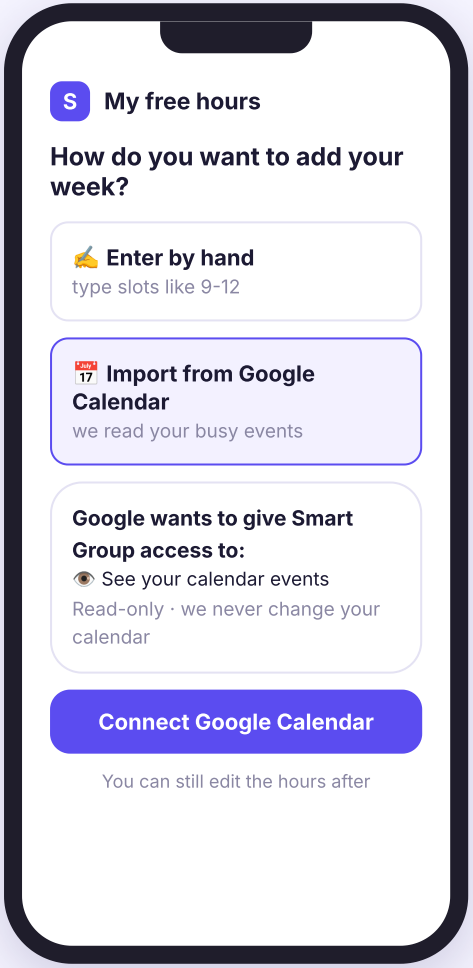
EVERYONE ADDS THEIR WEEK

- **Rosha** — 5 days entered ✓
- **Mohamed** — 5 days entered ✓
- **Farnam** — 5 days entered ✓
- **Borna** — 3 days entered ✓

5 GOOGLE CALENDAR IMPORT

Already have a calendar? Just connect it.

Typing every slot by hand takes time. With Google Calendar import, the app reads each member's busy events and turns the gaps between them into free hours. After that, the same scheduler finds the meeting.



📱 **Mohamed connects his calendar**

MOHAMED'S WEDNESDAY, FROM GOOGLE CALENDAR TO FREE HOURS



Free hours = the day (8:00–22:00) minus the busy events → `[(14, 19)]`, the same format as when you type it by hand.

HOW IT WORKS

1. Sign in with Google (read-only)
2. Read this week's busy events
3. Turn the gaps into free hours
4. The scheduler does the rest

MIX AND MATCH

- R **Rosha** · typed by hand
- M **Mohamed** · Google Calendar
- F **Farnam** · typed by hand
- B **Borna** · Google Calendar

6 FINDING THE MEETING TIME

No more "when are you free?"

The program compares everyone's free hours day by day. Two sorted lists are walked through with two pointers: the overlap goes from the later start to the earlier end. The longest slot of the week wins.

WHOLE GROUP'S WEEK

● = free all 4 free

	Mon	Tue	Wed	Thu	Fri
9:00	●●	●	●	●●	●●
10:00	●●●	●●	●	●●	●●●
11:00	All free ✓	●●	●	●	All free ✓
12:00	●●●	●●	●●	●	●●●
13:00	●	●	●●●	●	●●
14:00	●	●●	All free ✓	●	●
15:00	●	●●	All free ✓	●●	●
16:00	●●●	●●	All free ✓	●●	
17:00	All free ✓	●●	●●	●●	
18:00	●●●		●		
19:00	●●●				
20:00	●●				

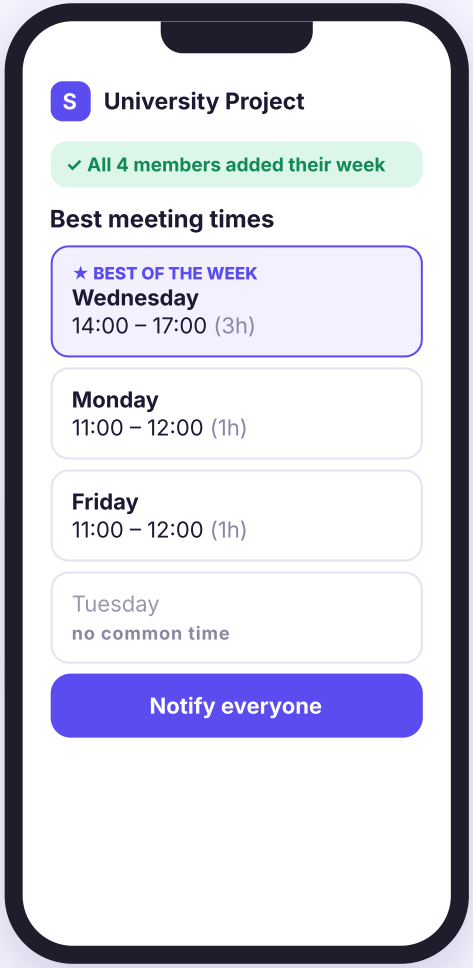
WEDNESDAY, STEP BY STEP · `FIND_COMMON_TIME()`

R	**Rosha** free 13–18	→ 13–18
M	**Mohamed** free 14–19	→ 14–18
F	**Farnam** free 9–17	→ 14–17
B	**Borna** free 12–17	→ 14–17

Tuesday and Thursday: Borna has no free hours → no common time.

```
option 4 · Suggest Meeting Time

Choose an option: 4
Day (b=Back, 0=Cancel): wednesday
Best meeting time: (14, 17)
```



📱 The app suggests times

7 NOTIFICATIONS

The meeting is set. Everyone knows.

As soon as the last member saves their free hours, the program picks the best meeting of the week and sends it to every member on the platforms they chose. If nothing changed, the same meeting is never sent twice.

Borna saves the last missing day

```
Save this day? (y/n, b=Back, 0=Cancel): y
Availability saved!
Best meeting time: Wednesday, (14, 17)
Email sent successfully to rosha@example.com!
Telegram message sent successfully!
[SIMULATED WHATSAPP] Hello Mohamed! ...
[SIMULATED SMS] Hello Farnam! ...
Email sent successfully to borna@example.com!
Notification process completed!
```

💻 **Our program today** · `simulated.py`

🔁 **No duplicates:** the meeting is saved as `last_notified_meeting`. If the result is the same next time: *"Notification already sent for this meeting!"*

✈️ Telegram — REAL

Hello Rosha!

Your group 'University Project' has scheduled a meeting.

Day: Wednesday
Time: 14:00 - 17:00

See you there!

R **Rosha**

🟢 WhatsApp — SIMULATED

Hello Mohamed!

Your group 'University Project' has scheduled a meeting.

Day: Wednesday
Time: 14:00 - 17:00

See you there!

M **Mohamed**

💬 SMS — SIMULATED

Hello Farnam!

Your group 'University Project' has scheduled a meeting.

Day: Wednesday
Time: 14:00 - 17:00

See you there!

F **Farnam**

✉️ Email — REAL

To: borna@example.com
Meeting Scheduled - University Project

Hello Borna!

Your group 'University Project' has scheduled a meeting.

Day: Wednesday
Time: 14:00 - 17:00

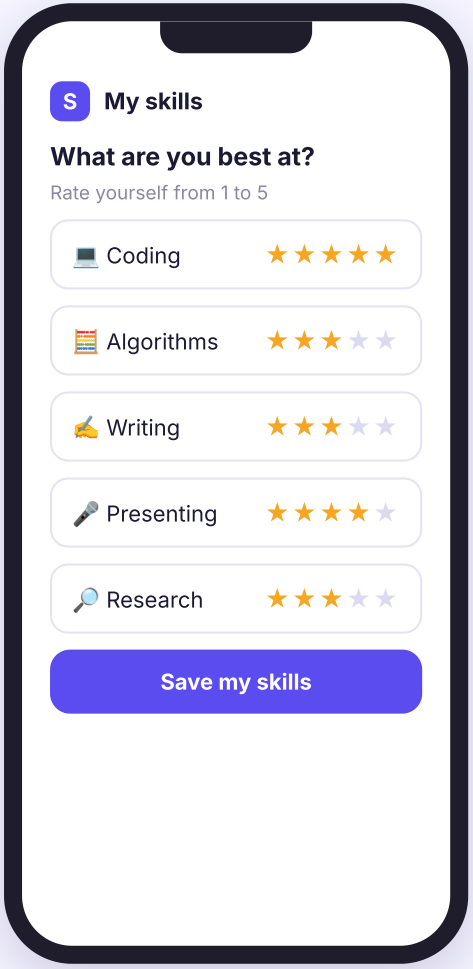
See you there!

B **Borna**

8 FAIR TASK DISTRIBUTION

Everyone does what they're best at, fairly

Our next step: each member rates their skills, and the app matches every task to the right person while keeping the workload balanced. New tasks are sent on the same platforms as the meetings.



📱 **Rosha rates her skills**

TASK BOARD · UNIVERSITY PROJECT

⚖️ BALANCED WORKLOAD

Assigned by the app · anyone can swap with a teammate

R **Build the Telegram and menu features**
Rosha · matched by Coding 5/5 — Workload 25%

M **Write the scheduling algorithm**
Mohamed · matched by Algorithms 5/5 — Workload 25%

F **Design the slides and present**
Farnam · matched by Presenting 5/5 — Workload 25%

B **Write the report and the test scripts**
Borna · matched by Writing 5/5 — Workload 25%

HOW TASKS ARE MATCHED

1️⃣ Everyone rates their skills
2️⃣ Each task goes to the best match
3️⃣ Nobody gets much more work than the others

✈️ Telegram · Farnam

🆕 New task assigned:
Design the slides and present
📅 Next meeting: Wednesday, 14:00 – 17:00

9 THE FULL EXPERIENCE

From chaos to a plan, in six steps

Everything is saved in data/groups.json, so the group can be loaded again next time. Here is what works today and what we would add next.

1 👥 **Create a group**
Name it and add members

→

2 🔔 **Pick platforms**
Email, Telegram, WhatsApp, SMS

→

3 🔐 **Connect Telegram**
One-time code to our bot

→

4 🕑 **Add free hours**
Whole hours, per day

→

5 🔍 **Find the time**
Two-pointer overlap, longest wins

→

6 📣 **Get notified**
Right message, right place

✅ WORKS TODAY

- Group, members and free hours, with back / cancel at every step
- Best meeting time for one day or the whole week
- Real Gmail and Telegram messages, verified Telegram accounts
- Automatic, no-duplicate notifications · save and load

🚀 NEXT IDEAS

- Half hours like 9:30, and one-off busy days (like exam days 📚)
- Real WhatsApp and SMS through a provider like Twilio
- Time zones for international groups 🌍
- 🚧 Google Calendar import (page 5)
- 🔮 Fair task splitting by skill (page 8)

Try it yourself

```
python main.py
```

Source code and setup: see the README on GitHub PythonHubb/smart-group-meeting-scheduler